Modul mahasiswa — Lab 01: model cloud, Git, dan API pertama

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca brief proyek kelompok. Bobot resmi tetap mengikuti RPS/LMS.

COMP6991031 · sesi 1. Kerjakan pada repo Git pribadi Lab 01. Bagian yang dijalankan adalah `starter/`; baca juga README lab dan panduan Git bersama. Gambar di modul ini adalah **contoh hasil starter**, bukan pengganti bukti pekerjaan Anda.

Tujuan dan teori ringkas

Setelah praktik, Anda dapat menjelaskan lima karakteristik cloud NIST, membedakan IaaS/PaaS/SaaS/BaaS/FaaS, memilih deployment public/private/hybrid/multi-cloud untuk suatu kasus, serta menunjukkan fungsi Git, GitHub, dan deployment Vercel. GitHub menyimpan kode dan riwayat sebagai SaaS; halaman Vercel adalah contoh PaaS dan `api/*.js` contoh FaaS. Model rekomendasi pada aplikasi adalah **alat diskusi**, bukan kunci jawaban.

Prasyarat

- Repo template meet1CloudService sudah disalin ke repo pribadi; Git terpasang dan `origin` menunjuk repo Anda. Jika belum, ikuti panduan Git.
- Node.js 20+ untuk jalur lokal, browser, dan terminal PowerShell atau Bash/Git Bash. Proyek ini tidak punya dependency eksternal, jadi **tidak perlu** `npm install`.
- Akun GitHub untuk push. Akun Vercel hanya untuk bagian deployment pilihan dosen; jalur lokal dapat dikerjakan tanpa akun cloud.
- Jangan masukkan NIM, token, password, atau data pribadi lain ke repo publik.

1. Periksa starter dan isi identitas aman

Dari **root repo Lab 01**, buka `starter/student.json`. Ganti tiga nilai contoh dengan nama, kode kelas, dan *username* GitHub Anda. Pertahankan JSON valid; jangan menambah field `nim`. Tes bawaan hanya memeriksa format, sehingga **18 tes bisa hijau saat file masih berisi contoh**. Dosen juga memeriksa isi nyata.

PowerShell:

```
Set-Location .\starter
node --version
npm test
```

Bash/Git Bash:

```
cd starter
node --version
npm test
```

Lab 01 · isi identitas lalu uji

```
$ Get-Content starter/student.json
{
  "name": "Nama Lengkap Anda",
  "class": "Kode Kelas",
  "github": "username-github"
}
```

```
$ Set-Location starter; npm test (ringkasan)
✓ kombinasi selalu berisi tepat satu model utama (2.3085ms)
✓ stack: tanggung jawab Anda tidak pernah naik dari kiri ke kanan (0.382ms)
✓ semua bobot hanya memakai factor yang terdaftar (0.8109ms)
✓ semua preset di cases.json memakai factor yang valid (0.5692ms)
✓ student.json valid dan field wajib terisi (19.5496ms)
ℹ tests 18
ℹ suites 0
ℹ pass 18
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 544.6951
```

Output aktual pada template; tiga nilai student.json masih contoh dan wajib diganti mahasiswa.

Perintah: `Get-Content starter/student.json` *dan* `npm test` *dari* `starter/`*. Fungsi: periksa tiga field identitas serta tes kode. Cara kerja: Node membaca JSON dan menjalankan 18 tes. Baca hasil:* `pass 18`*,* `fail 0` *pada salinan uji; gambar masih menunjukkan identitas contoh yang wajib Anda ganti. Ini render output command aktual, bukan tangkapan layar terminal mentah.*

Checkpoint 1: `npm test` menampilkan 18 tes lulus. Jika gagal, periksa tanda kutip/koma pada `student.json` dan nama field `name`, `class`, `github`.

PowerShell · labs/01/starter

```
npm test
✔ GET /api/hello: cold start hanya di panggilan pertama instance (24.4084ms)
✔ GET /api/recommend tanpa parameter mengembalikan petunjuk pemakaian (0.6105ms)
✔ GET /api/recommend dengan query string (0.9874ms)
✔ POST /api/recommend dengan body JSON (0.7829ms)
✔ POST dengan JSON rusak -> 400 (0.4687ms)
✔ POST tanpa factor valid -> 400 (0.3805ms)
✔ input kosong ditolak dengan pesan yang jelas (0.9422ms)
✔ normalizeFactors menerima array, objek, dan query string (0.905ms)
✔ W1 marketplace UMKM: public + BaaS/PaaS/FaaS (0.5483ms)
✔ W1 komponen pembayaran/email: cukup SaaS (0.196ms)
✔ W2 bank: analisis per komponen menghasilkan model berbeda (0.2974ms)
✔ data teregulasi selalu memunculkan catatan region Indonesia (0.1679ms)
✔ wajib bebas lock-in mendorong multi-cloud (0.1247ms)
✔ kombinasi selalu berisi tepat satu model utama (0.9626ms)
✔ stack: tanggung jawab Anda tidak pernah naik dari kiri ke kanan (0.2661ms)
✔ semua bobot hanya memakai factor yang terdaftar (0.2299ms)
✔ semua preset di cases.json memakai factor yang valid (0.1642ms)
✔ student.json valid dan field wajib terisi (1.7229ms)
ℹ tests 18
ℹ suites 0
ℹ pass 18
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 111.9897
```

- **Langkah:** Di `starter`, isi `student.json` lalu jalankan `npm test`. **Fungsi:** Menguji logika rekomendasi, API, dan format data mahasiswa sebelum demo. **Cara kerja:** Node menjalankan berkas tes dan membandingkan output fungsi/endpoint dengan hasil yang diharapkan. **Baca hasil:** Cari ringkasan `pass 18` dan `fail 0`; jika gagal, buka nama tes yang merah, bukan menyalin hasil contoh.

Baca ringkasan `pass 18` dan `fail 0` di bagian bawah. Screenshot ini berasal dari run contoh; ulangi tes di terminal Anda setelah mengisi `student.json`.

2. Jalankan halaman dan bandingkan tanggung jawab

Di terminal yang masih berada dalam `starter`:

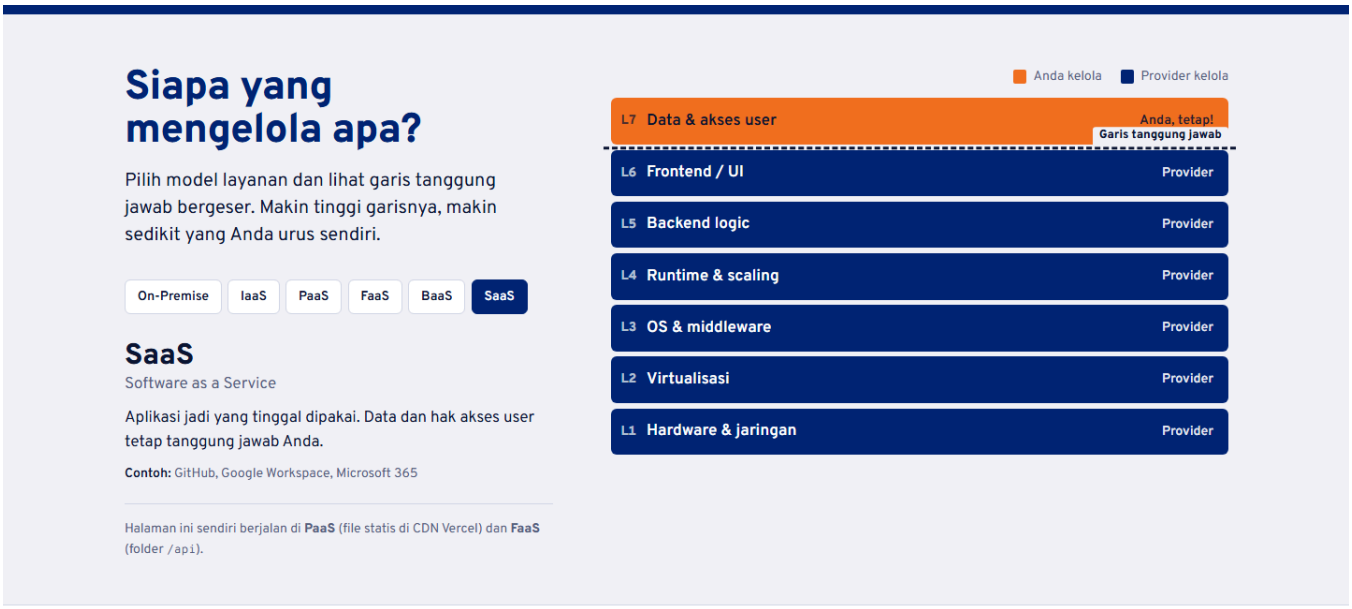
```
npm run dev
```

Buka `http://localhost:3000`. Klik IaaS, PaaS, SaaS, BaaS, dan FaaS, lalu catat untuk setiap pilihan siapa yang mengelola aplikasi, runtime, OS, dan perangkat fisik. Gambar berikut memperlihatkan **starter sebelum `student.json` diisi**; pada pekerjaan Anda, pesan identitas contoh harus hilang.



- **Langkah:** Jalankan `npm run dev`, lalu buka `http://localhost:3000` dan pilih model layanan pada web. **Fungsi:** Memperlihatkan batas tanggung jawab pengguna dan penyedia cloud. **Cara kerja:** JavaScript halaman mengganti lapisan diagram sesuai pilihan IaaS/PaaS/SaaS dan membaca faktor kasus. **Baca hasil:** Baca model terpilih, garis pembagi, serta pesan `student.json` contoh yang harus hilang setelah identitas diisi.

Checkpoint 2: halaman tampil dan garis tanggung jawab berpindah ketika model layanan diganti. Pilih dua preset studi kasus, lalu pilih faktor `spiky` dan `smallTeam` untuk kasus baru. Catat rekomendasi serta alasan yang ditampilkan.



- **Langkah:** Klik pilihan **SaaS** pada halaman Cloud Model Lab. **Fungsi:** Membandingkan tanggung jawab pengguna pada SaaS dengan PaaS atau IaaS. **Cara kerja:** UI memindahkan garis pembagi dan mewarnai lapisan sesuai model yang dipilih. **Baca hasil:** Periksa bahwa data dan akses pengguna tetap di sisi pengguna; jelaskan lapisan yang pindah ke penyedia.

Klik SaaS lalu bandingkan warna lapisan dan posisi garis dengan PaaS pada gambar sebelumnya. Pada gambar ini, lapisan L7 masih menjadi tanggung jawab pengguna.

3. Amati HTTP dan fungsi API

Biarkan server hidup. Buka terminal **kedua** di folder yang sama. Di PowerShell pakai `curl.exe`; di Bash pakai `curl`.

```
curl.exe -i http://localhost:3000/api/hello
curl.exe -i "http://localhost:3000/api/recommend?spiky=1&smallTeam=1"
curl.exe -i "http://localhost:3000/api/recommend?factors=ngawur"
```

```
curl -i http://localhost:3000/api/hello
curl -i 'http://localhost:3000/api/recommend?spiky=1&smallTeam=1'
curl -i 'http://localhost:3000/api/recommend?factors=ngawur'
```

Checkpoint 3: dua panggilan pertama memberi HTTP 200 dan JSON; faktor tak dikenal memberi HTTP 400. `hello` dapat menunjukkan `region: local` pada server lokal. Potongan respons berikut berasal dari endpoint yang benar-benar dijalankan; bagian JSON lain dipotong agar mudah dibaca.

Lab 01: cuplikan respons API nyata — keluaran aktual dari salinan uji


```
$ GET /api/recommend?spiky=1&smallTeam=1
{
  "ok": true,
  "factors": [
    "smallTeam",
    "spiky"
  ],
  "deployment": {
    "id": "public",
    "name": "Public cloud",
    "definition": "Infrastruktur milik provider, dipakai bersama banyak tenant lewat internet.",
    "score": 3,
    "reasons": [
      {
        "factor": "spiky",
        "label": "Traffic spiky, event-driven, atau sering idle",
        "weight": 2
      },
      {
        "factor": "smallTeam",
        "label": "Tim kecil, tanpa tim ops khusus",
        "weight": 1
      }
    ],
    "ranking": [
      {
        "id": "public",
        "name": "Public cloud",
        "score": 3
      },
      {
        "id": "hybrid",
        "name": "Hybrid cloud",
        "score": -1
      },
… (respons JSON dipotong untuk screenshot)
```

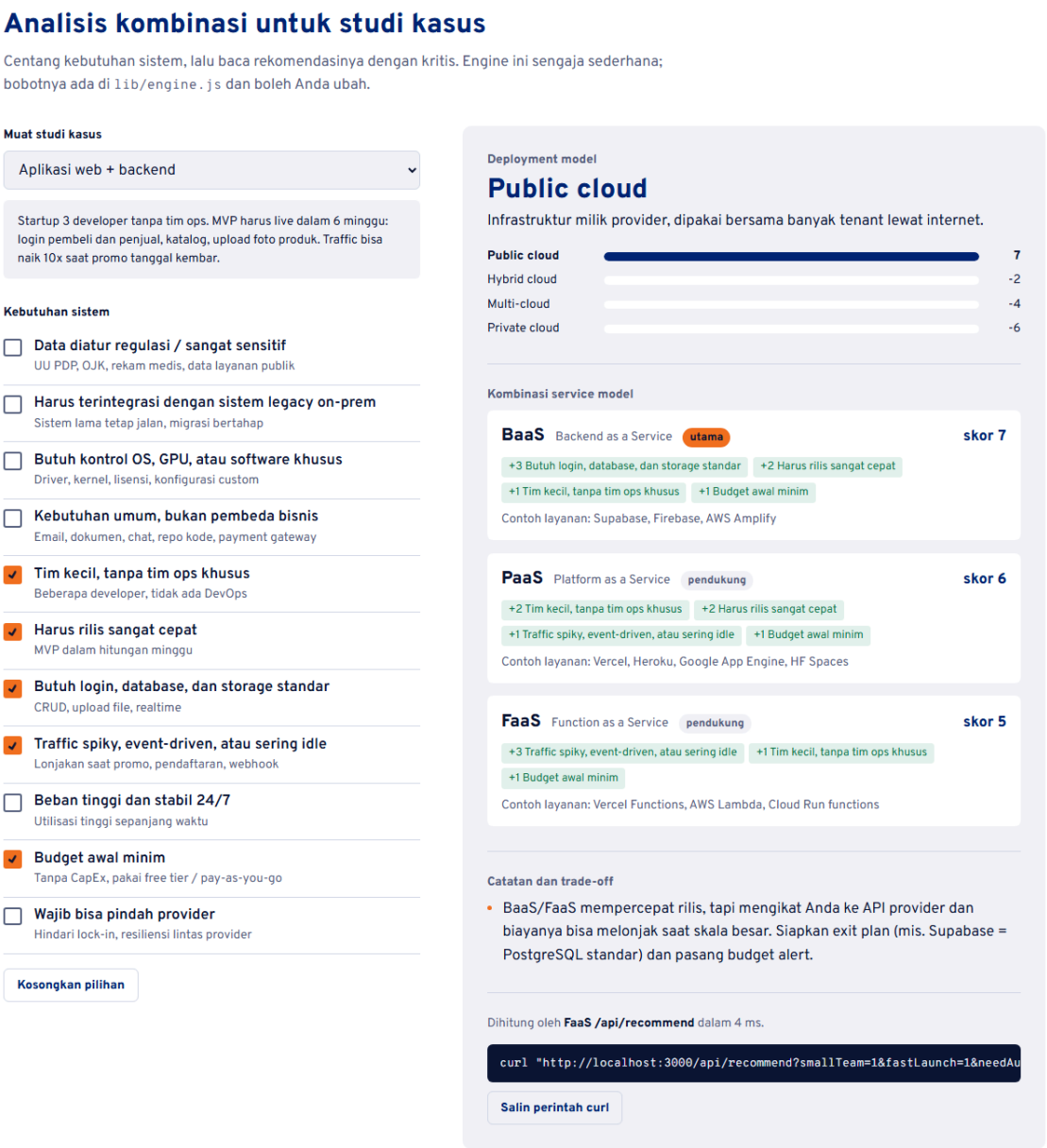



Perintah: GET `/api/hello`*, GET* `/api/recommend?spiky=1&smallTeam=1`*, lalu GET* `/api/recommend?factors=ngawur`*. Fungsi: membandingkan respons valid dan input yang ditolak. Cara kerja: server lokal memvalidasi query sebelum memberi JSON. Baca hasil: HTTP 200, 200, lalu 400; gambar adalah render output command aktual yang juga memuat ringkasan* `npm test`*.*

- **Langkah:** Jalankan `curl -i 'http://localhost:3000/api/recommend?spiky=1&smallTeam=1'`. **Fungsi:** Melihat respons API dari faktor skala dan ukuran tim. **Cara kerja:** Endpoint memvalidasi query, memberi skor model cloud, lalu mengirim status dan JSON alasan. **Baca hasil:** Baca HTTP 200, pilihan model, faktor, dan alasan; bandingkan dengan HTTP 400 untuk faktor tidak dikenal.

4. Analisis kasus dan jalur Git/cloud

Pilih **satu** kasus A–D dari case studies. Isi worksheet: pecah sistem menjadi komponen, pilih service/deployment model, berikan sedikitnya tiga alasan, tulis trade-off, lalu bandingkan dengan keluaran aplikasi. Di repo Lab 01 ini, gunakan `hasil/lab01.md` sebagai laporan dan tautkan worksheet atau salin analisisnya ke sana.



- **Langkah:** Di halaman web, pilih salah satu preset studi kasus dan jalankan rekomendasi. **Fungsi:** Menghubungkan kebutuhan kasus dengan model layanan/deployment. **Cara kerja:** Preset mengisi faktor; engine lokal menghitung rekomendasi dan trade-off yang ditampilkan di UI. **Baca hasil:** Cocokkan faktor tercentang dengan hasil public/private cloud serta alasan; tulis satu kondisi yang dapat mengubahnya.

Pada contoh ini preset memilih beberapa faktor dan engine menampilkan public cloud serta kombinasi BaaS/PaaS/FaaS. Tunjukkan faktor yang mendukung dan satu keadaan yang membuat rekomendasi ini perlu

ditinjau ulang.

Jika dosen meminta deployment, hubungkan repo pribadi ke Vercel dan atur **Root Directory** proyek menjadi `starter`; ikuti panduan Vercel asal. Periksa halaman dan kedua endpoint pada URL HTTPS yang diberikan Vercel. Jangan menganggap GitHub Pages menjalankan folder `api/`: Pages hanya menyajikan berkas statis. Perintah deploy dan pilihan paket layanan dapat berubah; gunakan dashboard dan dokumentasi resmi saat praktik.

Checkpoint 4: laporan menjelaskan keputusan **per komponen** dan menyatakan kapan rekomendasi engine perlu ditolak. Jika deploy dikerjakan, URL produksi membuka halaman dan endpoint API.

Pertanyaan yang dijawab di laporan

1. Bagian mana yang tetap Anda kelola ketika memakai PaaS dan FaaS pada proyek ini?
2. Apa beda public cloud dan multi-cloud? Apakah memakai GitHub dan Vercel otomatis membuat arsitektur multi-cloud?
3. Mengapa faktor `spiky` dan `smallTeam` bisa mengubah rekomendasi? Sebutkan satu kondisi nyata yang membuat rekomendasi tersebut kurang cocok.
4. Apa arti status 400 pada permintaan faktor tidak dikenal? Di mana validasi input dilakukan?
5. Mengapa repo private dan halaman Vercel publik bisa memiliki visibilitas berbeda?

Bukti, commit, dan push

Salin `hasil/TEMPLATE_LAPORAN.md` menjadi `hasil/lab01.md` **dari root repo**. Masukkan jawaban, simpan screenshot **milik Anda** untuk tes/UI/API di `hasil/bukti/`, serta tautan GitHub dan Vercel bila dipakai. Screenshot di modul ini hanya contoh bentuk hasil.

Kembali ke root repo, lalu jalankan:

```
git status --short
git add starter hasil/lab01.md hasil/bukti
git diff --cached --name-only
git diff --cached --check
git diff --cached
git commit -m "lab01: analisis model cloud dan API"
git push
```



SHA pada gambar adalah snapshot saat uji. Setelah modul diperbarui, jalankan ulang perintah untuk memeriksa commit terbaru.

Perintah: `git remote -v`, `git status --short`, `git log -1 --oneline`, `git rev-parse HEAD`, *dan* `git ls-remote origin refs/heads/main`. *Fungsi: memeriksa alamat repo, perubahan lokal, commit terakhir, dan hasil push. Cara kerja: SHA commit lokal dibandingkan dengan SHA branch* `main` *di GitHub. Baca hasil:* `Sama: True` *pada repo template pengajar; di repo pribadi Anda, SHA dapat berbeda tetapi SHA lokal dan remote harus cocok sesudah push. Ini render output command aktual.*

Periksa daftar dan isi berkas staged **secara lokal** sebelum commit: tidak boleh ada `.env`, token Vercel/GitHub, password, NIM, atau tangkapan layar yang memperlihatkan kredensial. Jika ini push pertama dan `git push` meminta upstream, gunakan `git push -u origin main` bila branch Anda bernama `main`. Workflow root `.github/workflows/verify.yml` mengulangi `npm test` saat push; tetap jalankan `npm test` lokal lebih dulu.

Berhenti dan mengatasi masalah

Hentikan `npm run dev` dengan `Ctrl+C` sebelum Lab 03 karena keduanya memakai port 3000 pada konfigurasi awal. Jika halaman 404, pastikan server dijalankan dari `starter`. Jika port 3000 sudah dipakai, hentikan layanan lain lebih dahulu. Jika tes hijau tetapi identitas masih contoh, edit `student.json` dan cek halaman lagi. Jika Vercel tidak menemukan repo, cek izin akses GitHub untuk repo tersebut.

Kunci praktik dan studi kasus

Kunci ini adalah **contoh jawaban yang dapat dipertanggungjawabkan**, bukan satu-satunya arsitektur. Pada pekerjaan sehari-hari, tim memilih layanan per komponen, menguji asumsi biaya/beban, dan mencatat tanggung jawab data. Jalankan aplikasi dahulu, lalu bandingkan keputusan sendiri dengan contoh ini.

1. Kode dan verifikasi dari repo Lab 01

Di root repo buka `starter/student.json`; isi `name`, `class`, dan `github` memakai identitas publik yang pantas. Contoh format aman (ganti nilainya):

```
{
  "name": "Mahasiswa Contoh",
  "class": "COMP6991031-A",
  "github": "akun-contoh"
}
```

Di PowerShell jalankan `Set-Location .\starter`; di Bash jalankan `cd starter`. Lalu:

```
npm test
npm run dev
```

`npm test` menguji 18 perilaku kode. Tes hijau **tidak** membuktikan data contoh sudah diganti; cek `student.json` dan halaman secara manual. Saat `npm run dev` masih hidup, buka terminal kedua di `starter/` dan gunakan `curl.exe` (PowerShell) atau `curl` (Bash):

```
curl -i http://localhost:3000/api/hello
curl -i 'http://localhost:3000/api/recommend?spiky=1&smallTeam=1'
curl -i 'http://localhost:3000/api/recommend?factors=ngawur'
```

Dua request pertama memberi HTTP 200 dan JSON. Faktor tidak dikenal ditolak HTTP 400; ini kesalahan input klien, bukan server crash. Baca implementasi `starter/api/recommend.js` dan `starter/lib/engine.js` untuk melihat validasi dan alasan rekomendasi. Pada PowerShell, tanda kutip tunggal URL juga dapat dipakai, tetapi gunakan `curl.exe` agar memanggil program curl asli.



Perintah: `npm test`*, lalu GET* `/api/hello`*, GET* `/api/recommend?spiky=1&smallTeam=1`*, dan GET faktor tak dikenal. Gambar adalah transkrip hasil nyata yang ditata ulang agar terbaca.* `pass 18/fail 0` *membuktikan tes kode; HTTP 200/200/400 membuktikan jalur web dan validasi input.*



Perintah: `npm run dev` *dari* `starter/`*, lalu buka* `http://localhost:3000`*. Server Node menyajikan HTML, JavaScript, dan endpoint lokal. Perhatikan bahwa gambar memakai identitas contoh; saat tugas, ganti* `starter/student.json` *dengan identitas publik Anda sebelum mengambil bukti sendiri.*

2. Kunci perbandingan model layanan

Model	Yang dikelola tim pada proyek ini	Contoh keputusan
IaaS	OS, runtime, aplikasi, data, dan akses di atas VM	Cocok saat kontrol OS atau software khusus diperlukan.
PaaS	Kode aplikasi, data, dan akses; runtime/platform dikelola penyedia	Halaman web cepat dirilis tanpa merawat OS server.
FaaS	Fungsi, data yang dipakai, dan izin pemanggilan	Handler `/api/recommend` berjalan saat request datang.
BaaS	Skema/data, aturan akses, dan integrasi SDK	Auth, database, dan storage dapat dibeli sebagai layanan.
SaaS	Data, konfigurasi, pengguna, dan hak akses	GitHub issue/repo atau layanan email siap pakai.

Pada semua model, klasifikasi data, pemberian hak akses, dan tujuan pemrosesan tetap harus diputuskan oleh pemilik aplikasi. Public cloud menjelaskan **lokasi/kepemilikan infrastruktur yang dipakai bersama**; multi-cloud berarti memakai lebih dari satu penyedia cloud untuk workload. Memakai GitHub untuk kerja tim dan Vercel untuk hosting belum otomatis berarti aplikasi produksi didesain untuk berjalan lintas dua cloud.

3. Kunci kasus A–D dari `starter/exercises/case-studies.md`

Kasus	Contoh rancangan per komponen	Tiga faktor utama	Trade-off dan cek berikutnya
A — event kampus	Form sederhana: SaaS; bila perlu alur unik, frontend PaaS + database/Auth BaaS + webhook FaaS pada public cloud	Ramai hanya beberapa hari, panitia berganti, dana terbatas	Ketergantungan vendor dan batas kuota; uji beban pendaftaran serta ekspor data setelah acara.
B — jaringan klinik	Pertimbangkan SaaS RME yang memenuhi kontrak/lokasi data; integrasi khusus dapat memakai PaaS atau private service terbatas	Data sensitif, 12 cabang, hanya dua staf IT	Vendor lock-in dan kegagalan koneksi; verifikasi ketentuan terbaru, audit akses, cadangan, serta kemampuan ekspor sebelum memilih.
C — AI dubbing	Training batch pada IaaS GPU yang dapat dipindah; inference pada container/PaaS terpisah; SaaS untuk fungsi umum	GPU dipakai beberapa hari, klien global, ingin pindah penyedia	Spot dapat terhenti dan perpindahan data mahal; gunakan checkpoint model dan ukur biaya transfer antarlokasi.
D — portal PPDB	Integrasi identitas/data internal dekat sistem pemda; frontend dan antrean elastis pada layanan yang memenuhi aturan lokasi/akses	Lonjakan dua minggu, sistem lama harus terhubung, data warga sensitif	Kompleksitas hybrid, ketergantungan jaringan, dan pemulihan; uji beban serta backup/restore, lalu verifikasi aturan pemerintah yang berlaku.

Untuk setiap pilihan, tulis **alternatif** yang Anda tolak dan alasannya. Misalnya pada kasus A, Google Forms cukup jika hanya mengumpulkan pendaftaran sederhana; ketika perlu kuota real-time, pembayaran, dan notifikasi, alur khusus mungkin diperlukan. Rekomendasi engine adalah titik awal diskusi: ia menghitung faktor, tetapi tidak mengetahui kontrak vendor atau batas kebijakan institusi Anda.

4. Kunci lima pertanyaan laporan

1. Pada PaaS tim masih mengelola kode aplikasi, data, konfigurasi, dan akses. Pada FaaS tim juga menulis fungsi serta mengatur izin/trigger; penyedia menjalankan infrastrukturnya.
2. Public cloud adalah model deployment pada infrastruktur penyedia yang dibagi secara terkontrol. Multi-cloud adalah penggunaan lebih dari satu penyedia untuk kebutuhan cloud; dua merek alat dalam proses kerja tidak otomatis membuktikan desain multi-cloud aplikasi.
3. `spiky` mendorong kapasitas elastis, sedangkan `smallTeam` mendorong layanan yang tidak perlu tim operasi besar. Bila data wajib tinggal di lingkungan tertentu atau biaya transfer dominan, rekomendasi public

PaaS/BaaS bisa kurang cocok.

4. HTTP 400 menandai query tidak sesuai kontrak API. Handler `starter/api/recommend.js` dan normalisasi faktor di `starter/lib/engine.js` menolak faktor tak dikenal sebelum memberi rekomendasi.
5. Visibilitas kode GitHub dan visibilitas hasil deploy adalah dua pengaturan berbeda. Repo dapat private sementara situs yang diterbitkan melalui URL produksi diatur public; cek izin proyek serta jangan memuat rahasia di bundle browser.

5. Git dan deployment pilihan

Kembali ke root repo dengan `cd ..`. Simpan analisis ke `hasil/lab01.md` dan bukti hasil sendiri ke `hasil/bukti/`, lalu ikuti panduan Git. Di GitHub, buat issue analisis kasus, lakukan commit yang menyebut nomor issue jika diperlukan, dan lihat tab Actions. Jika dosen meminta Vercel, impor **repo pribadi** dan set Root Directory ke `starter`; setelah deploy, buka halaman dan dua endpoint pada URL HTTPS produksi. Catat commit yang dideploy agar halaman lama dapat didiagnosis. Jalur GitHub/Vercel membutuhkan akun dan izin repo; jalur lokal di atas sudah cukup untuk mencoba seluruh kode aplikasi.